

PHP8時代に array型と向き合う

Faced with array type in the PHP 8.x era



pixiv Inc.
USAMI Kenta

pixiv

お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE / にゃんだーすわん
- ピクシブ株式会社 pixiv事業本部 エンジニア
 - 最近はピクシブ百科事典(dic.pixiv.net)を開発しています (PSR-15!)
- Emacs Lisper, PHPer
 - Emacs PHP Modeを開発しています (2017年-)
- PHPカンファレンス実行委員、PHPerKaigi コアスタッフ

tadsanのあれこれが読める場所



- <https://tadsan.fanbox.cc/>
- <https://scrapbox.io/php/>
- <https://www.phper.ninja/>
- <https://zenn.dev/tadsan>
- <https://qiita.com/tadsan>
- <https://github.com/bag2php>

WEB+DB PRESS総集編



Software Design 11月号



この記事は執筆時点で10月2～3日に開催を予定しているPHPカンファレンス2021と連動した短期集中連載の最終回です。つまり、この記事が掲載されたSoftware Design 2021年11月号をみなさんが手にとられるころにはすでに開催されています。カンファレンスで発表されたセッションはYouTubeチャンネル^{注1}にアーカイブが公開されているはずですので、興味のある方はぜひご覧ください。筆者はPHPカンファレンス2021で「配列、ジェネリクス、PHPで書けない型」と題したトークを発表します。

最終回では近年の動的言語の型検査の動向についての概観に触れたあと、PHPの動向について説明します。

動的型付き(statically typed)の言語と呼びます。事前にデータの種類を固定せずに処理することを動的型検査(dynamic type checking)と呼び、実行時にデータに応じた処理をすることが基本のプログラミング言語を動的プログラミング言語(dynamic programming language)と呼びます。

メジャーなものでは、C言語、Java、C++、C#、Go、Swift、Rust、Scala、Kotlin、Haskell、OCamlなどは静的型付きの言語です。また、Python、JavaScript、Ruby、シェルスクリプト、ClojureなどLisp系の言語、Smalltalk、そしてPHPは動的言語と呼ばれます。俗に実行ファイルを生成するものをコンパイル言語、ソースコードを直接実行するものをインタプリタ言語として

前回までのあらすじ

最後のPHP勉強会(2020年)

≡ PIXIV FANBOX



tadsan

クリエイターページ

投稿する

ダッシュボード

投稿管理

クリエイターページ管理

プラン管理

クリエイターID設定

支援者一覧

フォロワー一覧

おたより

支援金管理/振込

二次元コード発行

配列の型と向き合う

2020-02-26 PHP勉強会@東京

2020年2月27日 02:33

全体公開

arrayの型と向き合う #phpstudy

2020年最初のPHP勉強会@東京である第145回 PHP勉強会@東京で話してきました。話の経緯としては、この数日ざっくり書いている型について記事が一向に書き上げられなくてむしゃくし...



PHPカンファレンス沖縄2021

YouTube Premium JP 検索

▶ 4.44 ★★★★★ PHPCONF OKINAWA 2021

今日からできる安心型付け入門
Introduction of typing in PHP

2021-05-29 YouTube
PHPカンファレンス沖縄

Platinum Sponsor PHP 技術者認定機構 Toysub!

#phpcon_okinawa

PHPカンファレンス沖縄2021 Track A

2,107 回視聴 • 2021/05/29 にライブ配信

32 低評価 共有 オフライン 保存 ...

PHPカンファレンス沖縄
チャンネル登録者数 60人

チャンネル登録

Twitter feed:

- #phpcon_okinawa #track_a
今日も誰かのにちようび(おい... 14s
(でも「コミットの粒度」みたいな話をして気分良く会話が終わった経験がないかもな... 膝に矢を受けてしまった...)
- #phpcon_okinawa #track_a
たつきち フリーランス時々CT... 2m
コミットメッセージは
github.com/streamich/git-... 使って書くのが個人的にはおすすめです!
- #phpcon_okinawa
ボンちゃん@ビットラボWEB工... 3m
新たに知識を得る事は快感^ - ^
- #phpcon_okinawa #track_a
そーだい@初代ALF @soudai1025 5m
- Takuto Wada @t.wada
Replying to @hagevashi
Git のコミットコメントの規約としては、Why の分量が多い時はコミットのタイトル (1行目) には概要を書き、空行を挟んで3行目以降に Why を書いていくのがオススメです。

PHPカンファレンス2021

採択 2021/10/03 10:00～ Track2 Long session (60 mins)

配列、ジェネリクス、PHPで書けない型

PHP Conference Japan 2021

☆ 6



うさみけんた  [tadsan](#)

近年、PHPの機能強化により型宣言だけで安全に書ける範囲が広がっています。

その一方でPHPStanやPsalmといった静的解析ツールはPHPでは表現できない強力な型を提供することでコード品質向上の価値を高めることが可能です。

PhpStormはPHPプログラマに静的型の恩恵をもたらした一方、先述のツールと比べサポートする型について見劣りする点がありました。

ところが最近リリースされた待望の新バージョンである2021.2はこれまで静的解析ツールの専売特許だった型のいくつかをサポートされるようになり、型検査だけでなく入力補完などの恩恵を受けられるようになりました。そこで追加された型のひとつがジェネリクス（総称型）です。

本発表では、PHPの型についての基礎知識（[今日からできる安心型付け入門](#)）があることを前提として、[PHPにジェネリクスは入るのか？](#)の内容を軸に、PHPの型宣言では現時点で賄えない部分、特に配列の型およびジェネリクスの概念、class-string型の概念、そしてジェネリクスを実際に活用するためのテクニックを説明します。

Discord Channel: [#track2-4-php-type](#)

なんかいろいろ
話してきました

PHPの進化は 型宣言の進化

型がついていない関数(PHP5)

```
function add($a, $b) {  
    return $a + $b;  
}
```

スカラー型宣言(PHP7)

```
function add(int $a, int $b): int {  
    return $a + $b;  
}
```

int + intって
本当にintなの？

広い値をとるにはfloatが必要

ひとつの解決策ではあるが… 不必要にfloatを強制するのか

```
function add(float $a, float $b): float {  
    return $a + $b;  
}
```


PHPDocの型注釈(アノテーション)

```
/**
 * @param int|float $a
 * @param int|float $b
 * @return int|float
 */
function add($a, $b) {
    return $a + $b;
}
```

あえて型宣言を省略する

ユニオン型宣言 (PHP8.0)

```
function add(int|float $a, int|float $b): int|float {  
    return $a + $b;  
}
```

クラスの型も PHP7→8で進化

著者を持つBookクラス

```
class Book
{
    /** @var Author */
    private $author
    public function __construct(Author $author) {
        $this->author = $author;
    }
}
```


PHP 7.4でプロパティ型宣言

```
class Book
{
    /** @var Author */
    private Author $author
    public function __construct(Author $author) {
        $this->author = $author;
    }
}
```

コンストラクタプロモーション(PHP8)

```
class Book
{
    function __construct(
        private Author $author
    ) {
        // 何も書かなくてもプロパティに代入される
    }
}
```

readonlyプロパティ (PHP8.1)

```
class Book
{
    function __construct(
        public readonly Author $author
    ) {
        // 何も書かなくてもプロパティに代入される
    }
}
```


かくしてPHP
PHPDocの型が
不要な世界に

Bookクラスは著者を持つ

```
class Book
{
    function __construct(
        private Author $author
    ) {
        // 何も書かなくてもプロパティに代入される
    }
}
```

上司

「共著を想定できてる？」

Bookクラスは複数の著者を持つ

```
class Book
{
    function __construct(
        private array $authors
    ) {
        // 何も書かなくてもプロパティは代入され
    }
}
```

arrayにすることで
複数であることを表現

Authorクラスの
情報が減ってる...

著者一覧を出力したい

```
class Book
{
    function printAuthors($fp): void {
        foreach ($this->authors as $author) {
            fwrite($fp, $author->getName());
        }
    }
}
```

Book::\$authors : array
中身が何かわからない！

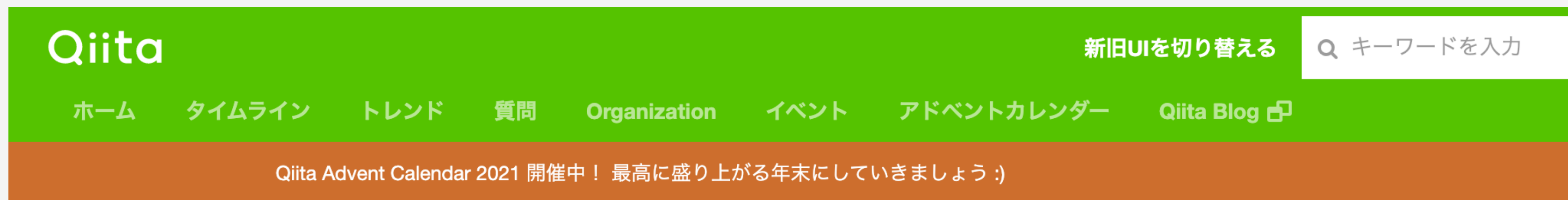
Authorクラスの
メソッドを補完できない

何か大事なものを
失ってしまった
気がする

今回は配列に絞って
話します

PHPの配列は万能

PHPの配列はなんでもできる



⚠ この記事は最終更新日から5年以上が経過しています。

@tadsan

投稿日 2016年08月12日 更新日 2016年08月19日 21279 views

PHPのarrayはすごいです！いろいろできます！奥深い！

PHP
LG TM

PHPの `array` はすごいです。いろいろできます。以前PHPの`list()`はタプル展開のための機能 - Qiitaといふ記事を書いて「配列はタプルだったんだよ！！！！」と力説したのですが、それ以外のデータ構造としての特徴について挙げていきます。

タイトルはてきとー (原題: 「All your data structure are belong to ARRAY.」 は All your base are belong to us - Wikipediaが元ねた。)

PHPマニュアルより引用

php

Downloads

Documentation

Get Involved

Help

php8.1

[Dutch PHP Conference 2022 - Call for Papers](#)

PHP マニュアル › 言語リファレンス › 型

Change language: Japanese ▼

[Submit a Pull Request](#) [Report a Bug](#)

配列

PHP の配列は、実際には順番付けられたマップです。マップは型の一種で、*値*をキーに関連付けます。この型は、さまざまな使い道にあわせて最適化されます。配列としてだけでなく、リスト (ベクター)、ハッシュテーブル (マップの実装の一つ)、辞書、コレクション、スタック、キュー等として使用することが可能です。PHP の配列には他の PHP 配列を値として保持することができるため、非常に簡単にツリー構造を表現することが可能です。

これらのデータ構造に関する説明は本マニュアルの範囲を超えるので省略しますが、各々について、少なくとも一つは例を示します。この分野は広範囲にまたがり、さまざまな文献が存在します。より詳細な情報については、これらの文献を参照ください。

PHPの配列は ハッシュテーブル

JavaScript/Rubyの場合

```
megurine % node
Welcome to Node.js v17.2.0.
Type ".help" for more information.
> a = []
[]
> a[1000] = 1
1
> a.length
1001
> a
[ <1000 empty items>, 1 ]
> █
```

PHPの場合

```
megurine % psysh
Psy Shell v0.11.0 (PHP 7.4.27 - cli) by Justin Hileman
>>> $a = []
=> []
>>> $a[1001] = 1
=> 1
>>> count($a)
=> 1
>>> $a
=> [
    1001 => 1,
]
>>> @$a[0]
<warning>PHP Notice:  Undefined offset: 0 in /Users/megurine/.composereval()'d code on line 1</warning>
=> null
>>> █
```

何が違うのか

他の言語における配列(array)

- C言語における配列はメモリ上の連続した領域に並べたもの
(ポインタとかアドレスとかの概念が絡んでくる)
- インデックスは0が始点、長さ(値の数)は最初に決める
- JavaScriptやRubyでの配列は複数の値を格納できるオブジェクト
 - インデックスは0が始点、動的に拡張される
 - 長さ以上のインデックスに代入されると、間の値はnull的な値で埋められる

PHPの配列はハッシュテーブル

- PHPの配列は基本的にハッシュテーブルというデータ構造
 - 任意の int または string の値をキーにできる
 - 長さ以上に代入しても間がnullで埋められることはない
- 連続していない前提の配列を特に「連想配列」と呼ぶ
 - 英語ではassociative array (略してassoc)などと呼ばれる
- PHPだけではなくLuaという言語も似た仕様

PHP7の内部実装から学ぶ性能改善テクニック

 slideshare
a Scribd company

HomeExplore

Search

Upload

This is the **last document** you can read this month as a guest. [Create your free account](#) to read unlimited documents.

PHP7の配列

- ❖ PHP7から配列と連想配列が別の構造になった
- ❖ 配列のときはハッシュテーブルが省略される
- ❖ 内部で自動的に選択される
- ❖ 配列・連想配列のデータは連続領域に置かれる

+

↓

< 57 of 64 >

⌂

Recommended



PHPバージョン別応答速度比較
Takayuki Saito



本当に怖いパフォーマンスが悪い実装...
Yahoo!デベロッパーネット...



C#の強み、或いは何故PHPから乗り換えるのか
Yoshifumi Kawai



PHP7から不変配列がOPcacheに乗るのでKV...
Yoshio Hanawa



自宅の消費電力をリアルタイムにグラフ化して...
Yoshio Hanawa



Zend VMにおける例外の実装
Yoshio Hanawa

PHP7から内部的に「配列」が入った

- 2015年リリースのPHP 7.0ではデータ構造に大きく手が入った
 - `$a = [1, 2, 3]` のような値は(C言語的な意味)の配列として保持される
 - 既存の配列に `$a['str'] = 'val'` のように追加するときハッシュテーブルに詰め替えられる

連想配列 vs リスト

- PHPでは区別されないが、たとえば値を`json_encode()`する時など連想配列と連続した値を厳密に比較したいことはありうる
 - JSONでは歯抜けの値があると `[]` ではなく `{}` になってしまう
 - `array_filter()` の結果など
- `array_values()` で歯抜けのない配列に変換できる
- PHP 8.1で `array_is_list()` で判定できる

array_is_list()について

- PHP 8.1で追加された関数
- 従来は 0 から順にforeachして歯抜けがないかチェックする必要があった
- 静的解析ツール(PHPStan, Psalmなど)ではPHPDocに `array<int,string>` の代わりに `list<string>` と書ける
 - 最近のPhpStormでは(検査は不十分だが)認識してくれる
 - Intelephenseは未実装

Authorsはどうか
よかったのか

Bookクラスは複数の著者を持つ

```
class Book
{
    function __construct(
        private array $authors
    ) {
        // 何も書かなくてもプロパティに代入される
    }
}
```

PHPDocの帰還

```
class Book
{
    function __construct(
        /** @var list<Authors> */
        private array $authors
    ) {
        // 何も書かなくてもプロパティに代入される
    }
}
```


配列はひとつ！じゃない！！

list<int>

array<int,string>

- ユーザIDがならんだリスト [123, 456]
- ユーザIDと名前の対応表 [123 => '野比のび太', 234 => '源静香']
- ユーザを表す構造体 ['id' => 123, 'name' => '野比のび太']
- ユーザを表す構造体がならんだリスト
[
 ['id' => 123, 'name' => '野比のび太'],
 ['id' => 234, 'name' => '源静香'],
]

array{id:int, name:string}

list<array{id:int, name:string}>